

1.智能体

1.什么是agent

在大模型应用开发中，智能体通常指一种以大语言模型为推理与决策核心，结合记忆、工具调用与环境交互能力，能够进行规划决策并执行复杂任务以达成目标的软件系统。

Agent的关键能力：

- 理解用户问题
- 如何拆解任务
- 判断是否需要工具
- 需要调用哪些工具
- 如何利用好工具结果生成回答&推进任务

2.Agent的核心组件

实际开发中几个要素并不需要同时出现，

- 必须的：行动（Action）
- 几乎总是存在的：工具（Tool）
- 有条件存在的：规划决策（Planning）
- 最容易被省略的：记忆（Memory）

3.Agent创建与调用

1) 历史上的调用

在 LangChain 0.x 时代，框架内的 Agent 系统经历了“碎片化”阶段。当时的设计理念是“针对场景设计特定 Agent”：

- 如果你要实现思维链推理（ReAct），就用 `create_react_agent` ；
- 如果需要结构化输出，就用 `create_structured_chat_agent` ；
- 要工具调用，则用 `create_tool_calling_agent` 。

举例：✗ v0.x 的复杂方式

```
In [ ]: # 需要多个步骤
from langchain.chat_models import init_chat_model
from langchain.agents import AgentExecutor, create_react_agent
from langchain_core.prompts import PromptTemplate
from dotenv import load_dotenv
import os

# 1. 模型初始化
load_dotenv(override=True)
load_dotenv()
model = init_chat_model(
```

```

model="deepseek-v4-flash",
model_provider="deepseek",
api_key=os.getenv("DEEPSEEK_API_KEY"),
# extra_body={
#     "thinking":{
#         "type": "disabled"
#     }
# }
)

# 2. 创建提示词模板
prompt = PromptTemplate.from_template("""
    You are a helpful assistant.

    Tools: {tools}
    Tool Names: {tool_names}

    {agent_scratchpad}
    """)

# 3. 创建 agent
agent = create_react_agent(
    llm=model,
    tools=tools,
    prompt=prompt
)

# 4. 创建 executor
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True
)

# 5. 调用
result = executor.invoke({"input": "问题"})

```

这种方式灵活，但也带来了三个明显问题：

- 1. 心智负担高——每种 Agent 都要单独记忆 API 与参数；
- 2. 可组合性差——多个 Agent 之间无法统一调度；
- 3. 生态碎片化——不同模块难以复用或协同演化。

2) 全新的调用

LangChain 在 1.0 版本后，团队做出了彻底重构：将所有 Agent 的创建方式统一为一个入口：

`create_agent()`。它取代了旧版本中的 `create_react_agent`、`create_json_agent`、`create_tool_calling_agent` 等多种分支函数，真正让开发者用一行代码即可创建任何类型的智能体。

同时在底层通过“中间件机制（Middleware）”和“标准模型接口（`invoke / stream`）”实现全局统一。这 让框架更轻、更稳，也更易于被集成到其他 Agent 平台中。

举例：✅ v1.x 的简洁方式：

```
In [ ]: from langchain.chat_models import init_chat_model
        from langchain.agents import create_agent

        # 1. 初始化模型
        model = init_chat_model("gpt-4o-mini", model_provider="openai")

        # 2. 创建 agent (一步完成)
        agent = create_agent(
            model=model,
            tools=[tool1, tool2],
            system_prompt="Agent 的行为指令" # 可选
        )

        # 3. 调用
        result = agent.invoke({
            "messages": [{"role": "user", "content": "问题"}]
        })
```

2.Agent的基本用法1：模型的传入方式

在 LangChain 1.2 中，`create_agent` 是构建智能体的核心方式，底层基于LangGraph 实现。

`create_agent` 完整参数：

```
In [ ]: from langchain.agents import create_agent

        agent = create_agent(
            model: str | BaseChatModel, # 必需：聊天模型
            tools: List[BaseTool], # 必需：工具列表
            *,
            system_prompt: str = "", # 系统提示词
            middleware: Sequence[AgentMiddleware[StateT_co, ContextT]] = (), # 中间件
            interrupt_before: List[str] = None, # 在某些工具前暂停（人机协作）
            interrupt_after: List[str] = None, # 在某些工具后暂停
            debug: bool = False, # 调试模式
            name: str | None = None, # 设置模型名称
        )
```

Agent在创建时，涉及到模型(Agent使用的模型)、可调用工具、系统提示词等参数的设置。

更多参数参考：

https://reference.langchain.com/python/langchain/agents/factory/create_agent

Agent中模型的传入方式：

本节我们只关注模型的传入。

模型是 Agent 的“大脑”，负责决策和推理。根据模型传入agent方式的不同，分为两种方式。

2.1 传入模型字符串

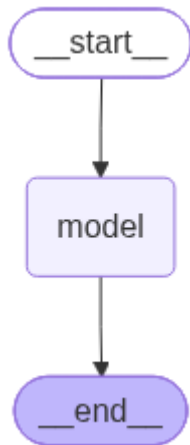
Agent根据传入的模型字符串，自主创建模型对象

```
In [1]: from langchain.agents import create_agent
from dotenv import load_dotenv

load_dotenv(override=True)
agent = create_agent("deepseek-v4-flash")
print(type(agent))

from IPython.display import Image, display
display(Image(agent.get_graph().draw_mermaid_png()))
```

```
c:\Users\huan.zheng\AppData\Local\miniconda3\envs\langchain1\Lib\site-
packages\langgraph\checkpoint\base\__init__.py:17: LangChainPendingDeprecationWarning:
The default value of `allowed_objects` will change in a future version. Pass an explicit value
(e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
<class 'langgraph.graph.state.CompiledStateGraph'>
```



由上可知，agent本质上是LangGraph的CompiledStateGraph实例，底层实现是一个图结构。

通过上述代码最后一行可以看到agent的图结构。

2.2 传入模型对象

```
In [3]: from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_deepseek import ChatDeepSeek
from dotenv import load_dotenv
import os

load_dotenv(override=True)
# 以ChatDeepSeek为例
# model = ChatDeepSeek(model="deepseek-v4-flash")
# 以init_chat_model为例
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
```

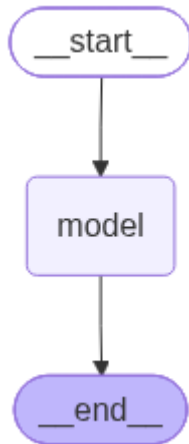


```

base_url=os.getenv("DEEPSEEK_BASE_URL")
)
agent = create_agent(model)
print(type(agent))
from IPython.display import Image, display
display(Image(agent.get_graph().draw_mermaid_png()))

```

<class 'langgraph.graph.state.CompiledStateGraph'>



3.Agent的基本用法2：如何调用Agent

agent.invoke() 是Agent 最基本的同步调用方法，它会阻塞程序执行直到返回最终结果。
具体的：

- **输入**：传入的参数为字典类型，字典内通过messages字段传递消息列表。即：“
{"messages":[{"role": "...", "content": "..."}]} ”
- **输出**：通过invoke调用Agent，底层可能会经历多轮交互，返回的是完整的消息列表，被封装在字典中，是messages字段的值。

```

In [ ]: response = agent.invoke({"messages": [...]})
# response 是字典类型
{
  "messages": [
    HumanMessage(...), # 用户问题
    AIMessage(...), # AI 工具调用
    ToolMessage(...), # 工具返回结果
    AIMessage(...) # 最终回答 ← 通常取这个
  ]
}

# 获取最终回答
final_answer = response['messages'][-1].content

```

举例1：

```

In [1]: from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os
load_dotenv(override=True)
from rich import print as rprint

```

```

# 以init_chat_model为例
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

agent = create_agent(model=model)
response = agent.invoke({"messages": ["你好"]}) # 默认是HumanMessage
print(type(response))
rprint(response)

```

c:\Users\huan.zheng\AppData\Local\miniconda3\envs\langchain1\Lib\site-packages\langgraph\checkpoint\base__init__.py:17: LangChainPendingDeprecationWarning: The default value of `allowed\_objects` will change in a future version. Pass an explicit value (e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.

```

from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
<class 'dict'>

```

```

{
  'messages': [
    HumanMessage(
      content='你好',
      additional_kwargs={},
      response_metadata={},
      id='12380fe9-6a03-423b-a457-d4ba0f0f1ebb'
    ),
    AIMessage(
      content='你好！很高兴见到你😊 有什么我可以帮你的吗？',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'We need answer in Chinese.
Need be careful. User says "你好" (hello). We
should respond friendly. Maybe introduce as AI assistant. Need not
overdo.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 49,
          'prompt_tokens': 84,
          'total_tokens': 133,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 34,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 84
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '33820836-bb32-4e25-a613-5bf96567d5f8',
        'finish_reason': 'stop',
        'logprobs': None
      },
      id='lc_run--019ff3d1-0f59-7350-878d-7d3940a949bb-0',
      tool_calls=[],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 84,
        'output_tokens': 49,
        'total_tokens': 133,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 34}
      }
    )
  ]
}

```

举例2：

invoke调用的核心就是输入一系列消息（messages），每条消息通常包含 role（如 "user", "assistant", "system", "tool"）和 "content"。

我们也可以在message列表的开头加入"system"角色的消息来定义Agent的行为。

```
In [2]: from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)
from rich import print as rprint

# 以init_chat_model为例
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

agent = create_agent(model)

resp = agent.invoke({
    "messages":[
        {"role":"system","content":"你是一个小学数学老师，耐心，幽默，讲解深入浅出"},
        {"role":"user","content":"100+50等于多少？"}
    ]
})

rprint(resp)
```

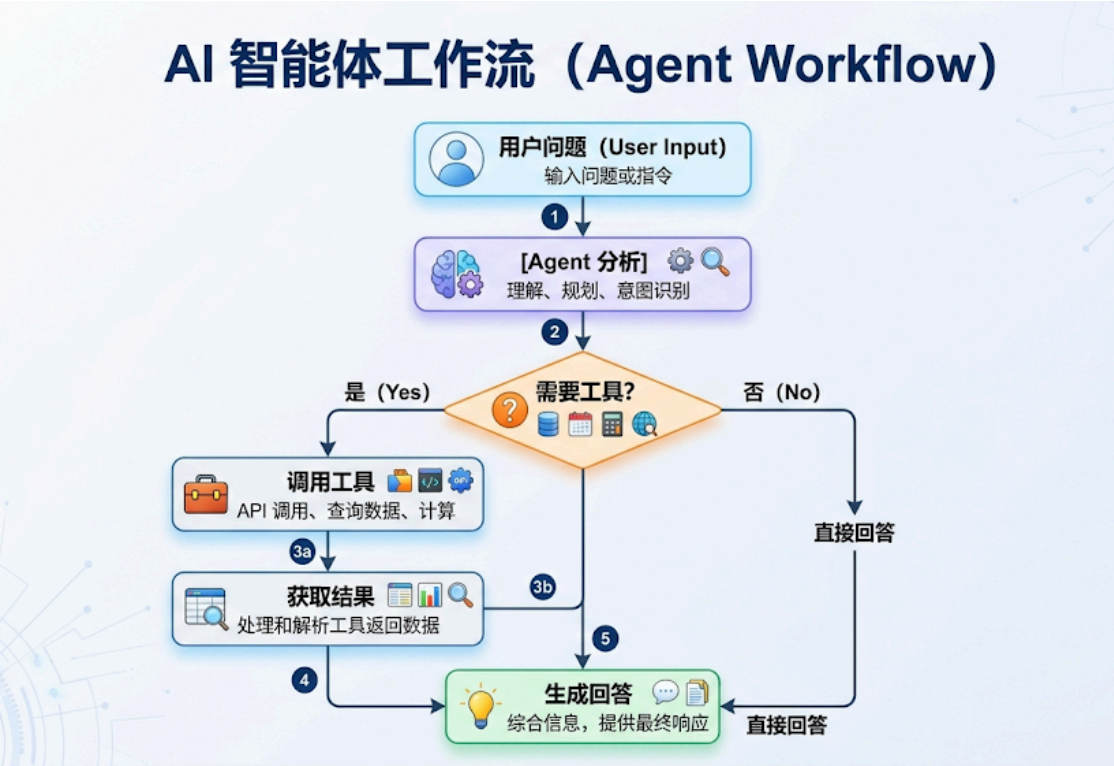
```

{
  'messages': [
    SystemMessage(
      content='你是一个小学数学老师，耐心，幽默，讲解深入浅出',
      additional_kwargs={},
      response_metadata={},
      id='2c53804b-4eba-4fda-b2da-b7dbef447777'
    ),
    HumanMessage(
      content='100+50等于多少？',
      additional_kwargs={},
      response_metadata={},
      id='6650bd5a-50de-4309-a3a5-e3ee9f164bad'
    ),
    AIMessage(
      content='100+50等于150呀，你看，就像你有100颗糖，再得到50颗，
      一共就有150颗糖，甜到心里去啦！😊',
      additional_kwargs={
        'refusal': None,
        'reasoning_content':
        '我们要求解100+50。这是简单的加法。答案150。注意用户问的是小学数学，但问题简单，
        直接回答即可。可以幽默一点。'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 67,
          'prompt_tokens': 102,
          'total_tokens': 169,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 32,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None},
        },
        'cached_tokens': 0,
        'prompt_cache_hit_tokens': 0,
        'prompt_cache_miss_tokens': 102
      },
      'model_provider': 'deepseek',
      'model_name': 'deepseek-v4-flash',
      'system_fingerprint':
      'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
      'id': 'dc3b161d-8453-42ac-adbe-f34f929c4dbe',
      'finish_reason': 'stop',
      'logprobs': None
    ),
    id='lc_run--019ff3d6-a865-76b2-81da-f74a59c813b8-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
      'input_tokens': 102,
      'output_tokens': 67,
      'total_tokens': 169,
      'input_token_details': {'cache_read': 0},
      'output_token_details': {'reasoning': 32}
    }
  ]
}

```

4.Agent的基本用法3：绑定工具

只有接入了一些工具，create_agent完成Agent创建才算完整。
Agent支持静态和动态绑定工具，后者需要用到中间件，后面会讲。在执行时：



这里的工具可以是LangChain内置的，也可以是自定义的。LangChain生态中已经内置集成了非常多的实用工具，开发者可以快速调用这些工具完成更加复杂工作流的开发。
LangChain内置工具列表：<https://docs.langchain.com/oss/python/integrations/tools>
其中典型的工具如下：

领域	工具名称	功能说明
搜索引擎与信息检索	DuckDuckGo Search	免费搜索引擎，返回网页标题、摘要与链接，常用于联网问答。
	Brave Search API	提供隐私友好的网页搜索结果，可配置输出深度与过滤。
	Bing Search Tool	微软 Bing 搜索接口，支持精确检索与多语言查询。
网页浏览与内容提取	Playwright Browser Toolkit	提供浏览器自动化访问、网页截图与文本抓取功能。
	Selenium Toolkit	类似于 Playwright，但偏向传统网页 DOM 操控。
生产力工具集成	Slack Toolkit	允许 Agent 在 Slack 中发送、读取或总结消息。
	Jira Toolkit	用于查询、创建或更新 Jira 项目工单。
	Google Drive / Docs Toolkit	操作 Google 文档、表格与文件搜索。

领域	工具名称	功能说明
数据库与数据操作	SQLDatabase Toolkit	连接 SQL 数据库，自动生成并执行 SQL 查询。
	Faiss / Chroma / Milvus VectorStore Tools	进行向量相似度搜索，用于 RAG 检索。
代码与终端执行	Python REPL Tool	执行 Python 代码片段并返回结果（沙盒环境）。
	Shell Tool (Bash)	允许执行 Shell 命令（需安全限制）。
	Requests Wrapper Tool	发起 HTTP 请求、API 调用，用于外部数据交互。
多模态工具	DALL-E Tool	生成图像，可结合文本描述创作内容。
	SerpAPI Image Search	图片搜索引擎，可返回图片 URL 与元信息。
	OpenAI Whisper Tool	音频转录工具，将语音文件转换为文本。
知识问答与嵌入检索	VectorStoreRetrieverTool	基于嵌入模型检索文档片段，用于 RAG 问答。
	MultiQueryRetrieverTool	生成多种查询变体以提升检索覆盖率。
云与AI服务	OpenAI Functions / Azure Functions Tool	让 Agent 调用 OpenAI 或 Azure 定义的函数。
	Anthropic ToolKit	集成 Claude 模型工具调用能力。
文件系统与数据处理	CSV Toolkit	读取、过滤与分析 CSV 文件数据。
	Pandas DataFrame Agent Tool	直接在 DataFrame 上执行查询与统计任务。

1) 基本用法

Agents支持绑定一或多个工具。

举例1：绑定一个工具

调用查询天气工具进行天气查询：

```
In [4]: from langchain.agents import create_agent
from langchain.tools import tool
from rich import print as rprint

@tool(parse_docstring=True)
def get_weather(city: str) -> str:
    """
    天气查询工具

    Args:
```

```

    city:城市名称
    """
    return f"{city}的天气为晴朗，25摄氏度"

agent = create_agent(
    model = model,
    tools = [get_weather]
)

resp = agent.invoke({
    "messages":[
        {"role":"system","content":"你是一个天气查询助手，只回答天气相关的问题，其他问题请直接回答：我不清楚这问题答案。"},
        {"role":"user","content":"上海的天气怎么样？"},
        #{"role":"user", "content": "100+50等于多少？"}
    ]
})

rprint(resp)

```



```

{
  'messages': [
    SystemMessage(
      content='你是一个天气查询助手，只回答天气相关的问题，其他问题请直接回答：我不清楚这问题答案。',
      additional_kwargs={},
      response_metadata={},
      id='a8fed6f7-ee8d-4f23-a257-46cfcde4b9ce'
    ),
    HumanMessage(
      content='上海的天气怎么样？',
      additional_kwargs={},
      response_metadata={},
      id='05b4eb55-58d2-438b-9ab9-e8c402d7148d'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'The user is asking about the weather in Shanghai. This is a weather-related question, so I should use the weather tool. Let me query the weather for Shanghai.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 78,
          'prompt_tokens': 384,
          'total_tokens': 462,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 33,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None},
          'cached_tokens': 0,
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 384
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint': 'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '8c796141-1eae-404b-9a88-35fff8bdee5d',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff3e2-a0c7-71a0-a83e-13840483897a-0',
      tool_calls=[
        {
          'name': 'get_weather',
          'args': {'city': '上海'},
          'id': 'call_00_BLnMZlj5urreVyRHdHib0648',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 384,
        'output_tokens': 78,
        'total_tokens': 462,
        'input_token_details': {'cache_read': 0},

```

```

        'output_token_details': {'reasoning': 33}
    },
    ToolMessage(
        content='上海的天气为晴朗, 25摄氏度',
        name='get_weather',
        id='933d0816-4bb8-441e-99dd-7ee51c1b0b64',
        tool_call_id='call_00_BLnMZlj5urreVyRHdHib0648'
    ),
    AIMessage(
        content='上海目前天气晴朗, 气温为25摄氏度。',
        additional_kwargs={'refusal': None, 'reasoning_content':
''},
        response_metadata={
            'token_usage': {
                'completion_tokens': 11,
                'prompt_tokens': 481,
                'total_tokens': 492,
                'completion_tokens_details': {
                    'accepted_prediction_tokens': None,
                    'audio_tokens': None,
                    'reasoning_tokens': 0,
                    'rejected_prediction_tokens': None
                },
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256},
                'prompt_cache_hit_tokens': 256,
                'prompt_cache_miss_tokens': 225
            },
            'model_provider': 'deepseek',
            'model_name': 'deepseek-v4-flash',
            'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
            'id': 'd6e26a29-5605-45ca-bb06-6b29d73bf2d1',
            'finish_reason': 'stop',
            'logprobs': None
        },
        id='lc_run--019ff3e2-a466-7283-ad1c-34c91e150a62-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 481,
            'output_tokens': 11,
            'total_tokens': 492,
            'input_token_details': {'cache_read': 256},
            'output_token_details': {'reasoning': 0}
        }
    )
}
]
}

```

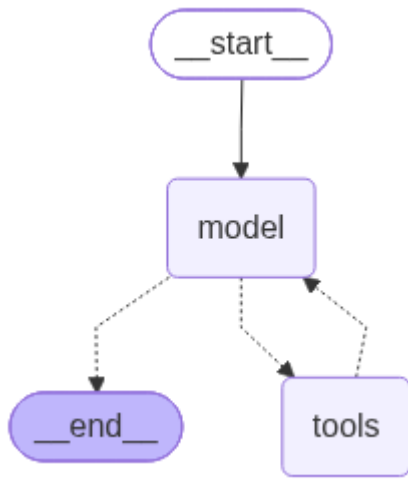
进一步：

```

In [5]: from IPython.display import Image, display

display(Image(agent.get_graph().draw_mermaid_png()))

```



举例2：接入内置工具

绑定内置的TavilySearch搜索工具，可以借助Tavily进行网络搜索和信息爬取。

这里我们需要先在tavily官网注册并获得API-KEY（每月有免费额度）：

<https://www.tavily.com/>。

```
In [6]: from langchain_tavily import TavilySearch
from dotenv import load_dotenv
import os

load_dotenv(override=True)
web_search = TavilySearch(
    tavily_api_key=os.getenv("TAVILY_API_KEY"),
    max_results=2
)

#这是一个高度封装的网络搜索工具，可以直接调用：
web_search.invoke("请问2026年足球世界杯有哪些参赛队？")
```

```

Out[6]: {'query': '请问2026年足球世界杯有哪些参赛队?',
'follow_up_questions': None,
'answer': None,
'images': [],
'results': [{'url':
'https://baike.baidu.com/item/2026%E5%B9%B4%E5%9B%BD%E9%99%85%E8%B6%B3%E8%81%94%E4%B8%96%E7%95%8C%E6%9D%AF/62863018',
'title': '2026年国际足联世界杯',
'content': '| 地区 | 球队 | 出线日期 | 参赛 次数 含2026 | 最好成绩 （年份） | \n --- --- \n | 东道主 (Host, 3支球队) | 美国国家男子足球队 | 2018年6月13日 | 12 | 季军 (1930) | \n | 加拿大国家男子足球队 | 2018年6月13日 | 3 | 小组赛 (1986、 2022年2次) | \n | 墨西哥国家男子足球队 | 2018年6月13日 | 18 | 八强 (1970、1986) | \n | 欧洲足球协会联盟 （UEFA, 16支球队） | 英格兰男子足球代表队 | 2025年10月15日 | 17 | 冠军 (1966) | \n | 法国国家男子足球队 | 2025年11月14日 | 17 | 冠军 (1998、2018 共2次) | \n | 克罗地亚国家男子足球队 | 2025年11月15日 | 7 | 亚军 (2018) | \n | 葡萄牙国家男子足球队 | 2025年11月16日 | 9 | 季军 (1966) | \n | 挪威国家男子足球队 | 2025年11月17日 | 4 | 十六强 (1998) | [...] | 德国国家男子足球队 | 2025年11月18日 | 21 | 冠军 (1954、1974、1990、2014 共4次) | \n | 荷兰国家男子足球队 | 2025年11月18日 | 12 | 亚军 (2010) | \n | 比利时国家男子足球队 | 2025年11月19日 | 15 | 季军 (2018) | \n | 奥地利国家男子足球队 | 2025年11月19日 | 8 | 季军 (1954) | \n | 西班牙国家男子足球队 | 2025年11月19日 | 17 | 冠军 (2010) | \n | 瑞士国家男子足球队 | 2025年11月19日 | 13 | 八强 (1934、1938、 1954) | \n | 苏格兰男子足球代表队 | 2025年11月19日 | 9 | 小组赛 | \n | 瑞典国家男子足球队 | 2026年4月1日 | 13 | 亚军 (1958) | \n | 土耳其国家男子足球队 | 2026年4月1日 | 3 | 季军 (2002) | \n | 捷克国家男子足球队 | 2026年4月1日 | 2 8+2 | 小组赛 (2006) 亚军 (1962、1934) | [...] | 突尼斯国家男子足球队 | 2025年9月8日 | 7 | 小组赛 (2022 等6次) | \n | 埃及国家男子足球队 | 2025年10月9日 | 4 | 埃及 (2018等4次) | \n | 阿尔及利亚国家男子足球队 | 2025年10月10日 | 5 | 十六强 (2014) | \n | 加纳国家男子足球队 | 2025年10月13日 | 5 | 八强 (2010) | \n | 佛得角国家男子足球队 | 2025年10月14日 | 1 | 首次 | \n | 南非国家男子足球队 | 2025年10月15日 | 4 | 小组赛 (2010等3次) | \n | 科特迪瓦国家男子足球队 | 2025年10月15日 | 4 | 小组赛 (2014等3次) | \n | 塞内加尔国家男子足球队 | 2025年10月15日 | 4 | 八强 (2002) | \n | 刚果民主共和国国家男子足球队 | 2026年4月1日 | 2 | 小组赛 (1974) | \n | 中北美洲及 加勒比海足球协会 (CONCACAF 3支球队) | 巴拿马国家男子足球队 | 2025年11月19日 | 2 | 小组赛 (2018) | ',
'score': 0.8934678,
'raw_content': None,
'id': 'f0d4e9-00'},
{'url':
'https://zh.wikipedia.org/wiki/2026%E5%B9%B4%E5%9C%8B%E9%9A%9B%E8%B6%B3%E5%8D%94%E4%B8%96%E7%95%8C%E7%9B%83',
'title': '2026年國際足總世界盃',
'content': '| FIFA World Cup 26 Copa Mundial de la FIFA 2026 Coupe du Monde FIFA 2026 | \n | We Are 26 Somos 26 Nous Sommes 26 | \n | 賽事資料 | \n | 主辦國家／地区 | 加拿大 墨西哥 美國 | \n | 比賽日期 | 6月11日–7月19日 | \n | 參賽隊數 | 48 隊（來自6個大洲） | \n | 球場 | 16 個（位於16個城市） | \n | 最終成績 | \n | 冠軍 | 西班牙（第 2 次奪冠） | \n | 亞軍 | 阿根廷 | \n | 季軍 | 英格蘭 | \n | 殿軍 | 法國 | \n | 賽事統計 | \n | 比賽場數 | 104 場 | \n | 總入球數 | 308 球（場均 2.96 球） | \n | 入場人數 | 6,810,966 人（場均 65,490 人） | \n | 最佳射手 | 法國 基利安·麥巴比（10 球） | \n | 最佳球員 | 西班牙 羅德里 | \n | 最佳守門員 "守門員 (足球)" | 西班牙 烏奈·西蒙 | \n | 最佳年輕球員 | 西班牙 保·庫巴西 | [...] 本屆賽事首次由過往的32支参赛球队扩大至48支。作为主办国，三个东道主自动获得决赛周参赛资格。在入围名单中，佛得角、库拉索、约旦和乌兹别克斯坦均为首次晋身世界盃決賽周。本屆賽事的冠軍為西班牙。 \n \n ## 擴充 \n \n [编辑] \n \n 早在2013年，時任歐足聯主席米歇爾·普拉蒂尼就提出了世界盃擴充的構想。2016年，國際足聯主席基安尼·恩芬天奴也再次提出類似觀點。但有反對人士認為目前賽制的比賽場次已經過多，擴充將稀釋比賽的可觀性及含金量，並指責因凡蒂諾向更多國家承諾世界盃參賽權，背後動機是主席選舉拉票的政治因素而非體育考量。 \n \n 自本屆賽事起，世界盃決賽圈參賽隊伍擴充至48支，較此前七屆增加了16支。按國際足協委員會在2023年3月14日通過的方案，球隊將被分為12個小組，每組4隊，每組前兩名與8個成績最好的小組第三名將晉級全新的32強淘汰賽階段，這將是自1998年以來世界盃首次擴軍並更改賽制。 \n \n 比賽總場次將從64場增至104場，進入四強的球隊比賽總場次將從7場增至8場。整個賽事將持續39天，比2014年和2018年賽事的32天有所增加。每支球隊仍將進行3場小組賽。 [...] 出線球隊 \n \n 被淘汰的球隊 \n \n 取消比賽資格或自動退出比賽 \n \n 非國際足協會員 \n \n 48支取得參賽資格的球隊中，有26支也參加了2022年世界盃。其中： \n \n 佛得角、古拉索、約旦和烏茲別克均為首次晉級世界盃決賽圈。 \n 剛果民主共和國和海地在1974年唯一一次參賽後重返世界盃。 \n 奧地利、挪威和蘇格蘭在1998年後重返世界盃。 \n 南非自2002年以來首次通過預選賽晉級，它們在2010年作為東道主自動獲得參賽資格。 \n 土耳其自2002年獲得季軍後首次獲得參賽資格。 \n 捷克自2006年以來首次獲得參賽資格。 \n 上屆東道主卡塔

```

爾首次透過预选赛晉級世界盃決賽圈。\\n 值得一提的是，意大利在欧洲区外围赛附加赛决赛点球大战中输给波斯尼亚和黑塞哥维那，错失参赛资格，成为首支连续三届无缘世界杯决赛圈的前冠军。\\n 俄罗斯由於入侵烏克蘭，继续被取消参加此次外圍賽及決賽週。\\n\\n 以下是按地区划分的出线球队名单，括號內的數字代表賽前的FIFA 男子世界排名。';

```
'score': 0.88420963,
'raw_content': None,
'id': '78c9bf-01'}],
'response_time': 1.08,
'request_id': '8be2c33d-ee3f-4403-bb9a-0fcdfb868419'}
```

可以直接帶入create_agent中作为外部工具。

```
In [8]: from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_tavily import TavilySearch
from dotenv import load_dotenv
import os
from rich import print as rprint

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

# 2. 工具实例化
web_search = TavilySearch(
    tavily_api_key=os.getenv("TAVILY_API_KEY"),
    max_results=2)

# 3. 创建Agent
agent = create_agent(
    model=model,
    tools=[web_search],
    system_prompt="你是一名多才多艺的智能助手，可以调用工具帮助用户解决问题。"
)

# 4. 运行Agent 获得结果
result = agent.invoke({
    "messages": [
        {"role": "user", "content": "请帮我查询2024年诺贝尔物理学奖得主是谁？"}
    ]
})

rprint(result)
# print(result['messages'][-1].content)
```

```

{
  'messages': [
    HumanMessage(
      content='请帮我查询2024年诺贝尔物理学奖得主是谁？',
      additional_kwargs={},
      response_metadata={},
      id='74cf79ca-ae96-4054-82b3-b14741a738d4'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'The user is asking about the
2024 Nobel Prize in Physics winners. Let me
search for this information.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 76,
          'prompt_tokens': 1918,
          'total_tokens': 1994,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 22,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 1792},
          'prompt_cache_hit_tokens': 1792,
          'prompt_cache_miss_tokens': 126
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'fc31a863-1d52-40fc-9ac9-501da178cc18',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff3ed-e65f-76f0-add1-ad68b0b4f674-0',
      tool_calls=[
        {
          'name': 'tavily_search',
          'args': {'query': '2024年诺贝尔物理学奖得主是谁'},
          'id': 'call_00_qWvPd4c8IiT9XdtWJf2V6258',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 1918,
        'output_tokens': 76,
        'total_tokens': 1994,
        'input_token_details': {'cache_read': 1792},
        'output_token_details': {'reasoning': 22}
      }
    ),
    ToolMessage(
      content='{"query": "2024年诺贝尔物理学奖得主是谁",
"follow_up_questions": null, "answer": null,
"images": [], "results": [{"url":
"https://news.sciencenet.cn/htmlnews/2024/10/531146.shtm", "title":

```

"刚刚，2024年诺贝尔物理学奖揭晓！ - 新闻- 科学网", "content": "| | |\n\n| |\n| |\n| |\n"

刚刚，2024年诺贝尔物理学奖揭晓！ |\n|

|\n\n\n北京时间10月8日下午5点45分许，2024年诺贝尔物理学奖揭晓。美国普林斯顿大学科学家约翰·霍普菲尔德（John J.

Hopfield）和加拿大多伦多大学科学家杰弗里·辛顿（Geoffrey E.

Hinton）获奖，以表彰他们“基于人工神经网络实现机器学习的基础性发现和发明”。

\n\n\n他们用物理学训练人工神经网络\n\n\n

今年的两位诺贝尔物理学奖得主使用物理学的工具来开发方法，这些方法是当今强大的机器学习的基础。John J.

Hopfield创造了一种联想记忆，可以存储和重建图像和其他类型的数据模式。Geoffrey

Hinton发明了一种方法，可以自动发现数据中的属性，从而执行识别图片中的特定元素等任务。 [...] | | | | | | | |

刚刚，2024年诺贝尔物理学奖揭晓！ | | |

北京时间10月8日下午5点45分许，2024年诺贝尔物理学奖揭晓。美国普林斯顿大学科学家约翰·霍普菲尔德（John J.

Hopfield）和加拿大多伦多大学科学家杰弗里·辛顿（Geoffrey E.

Hinton）获奖，以表彰他们“基于人工神经网络实现机器学习的基础性发现和发明”。 他们用物理学训练人工神经网络

今年的两位诺贝尔物理学奖得主使用物理学的工具来开发方法，这些方法是当今强大的机器学习的基础。John J.

Hopfield创造了一种联想记忆，可以存储和重建图像和其他类型的数据模式。Geoffrey

Hinton发明了一种方法，可以自动发现数据中的属性，从而执行识别图片中的特定元素等任务。 [...] | | | |

作者：冯丽妃，梅进 来源：科学网微信公众号 发布时间：2024/10/8 17:52:45 选择

字号：小 中 大 | | | | | | | |

刚刚，2024年诺贝尔物理学奖揭晓！ | | |

北京时间10月8日下午5点45分许，2024年诺贝尔物理学奖揭晓。美国普林斯顿大学科学家约翰·霍普菲尔德（John J.

Hopfield）和加拿大多伦多大学科学家杰弗里·辛顿（Geoffrey E.

Hinton）获奖，以表彰他们“基于人工神经网络实现机器学习的基础性发现和发明”。 他们用物理学训练人工神经网络

今年的两位诺贝尔物理学奖得主使用物理学的工具来开发方法，这些方法是当今强大的机器学习的基础。John J.

Hopfield创造了一种联想记忆，可以存储和重建图像和其他类型的数据模式。Geoffrey

Hinton发明了一种方法，可以自动发现数据中的属性，从而执行识别图片中的特定元素等任务。", "score": 0.92074,

"raw_content": null, "id": "926929-00"}, {"url":

"http://www.news.cn/world/20241008/c5aff4c9f7564a4c96d80d714fba74c8/c.html", "title":

"两名科学家因机器学习方面的贡献分享2024年诺贝尔物理学奖", "content":

"瑞典皇家科学院当天发表公报说，今年的两位诺贝尔物理学奖得主使用物理学工具，为当今强大的机器学习技术奠定了基础。约翰

·霍普菲尔德创建了一种联想记忆方法，可以存储和重构图像或其他类型的数据模式。杰弗里·欣顿发明了一种可以自动发现数据中

属性的方法，可用于识别图片中的特定元素等任务。 \n\n\n\u2003\u2003诺贝尔物理学委员会主席埃伦·穆恩斯在当天的新闻发布会

上表示，两名获奖者利用统计物理的基本概念设计了人工神经网络，构建了机器学习的基础。

相关技术已被用于推动多个领域的研

究，包括粒子物理、材料科学和天体物理等，也已用于日常生活中的人脸识别和语言翻译等。

她同时警告说，机器学习的快速发展

也引发了人们对未来的担忧，人类有责任以安全且道德的方式使用这项新技术。 \n\n\n10月

8日，在瑞典斯德哥尔摩举行的2024年诺

贝尔物理学奖公布现场，屏幕显示奖项得主美国普林斯顿大学的约翰·霍普菲尔德和加拿大多伦多大学的杰弗里·欣顿。新华社记者

彭子洋

摄\\n\\n\\n\u2003\u2003\u2003约翰·霍普菲尔德1933年出生于美国芝加哥，1958年获得美国康奈尔大学博士学位，现任美国普林斯顿大学教

授。", "score": 0.9161773, "raw_content": null, "id": "960ba0-01"}], "response_time": 0.0, "request_id":

"b75b7e20-193b-4efa-a3a9-77ab36b05d1a"}',

name='tavily_search',

id='020a54ab-fe40-4390-8754-2b62e5350fd6',

tool_call_id='call_00_qWvPd4c8IiT9XdtWJf2V6258'

),

AIMessage(

content='我查询到了2024年诺贝尔物理学奖的获奖信息，以下是详细内

容：\\n\\n## 🏆

2024年诺贝尔物理学奖\\n\\n**获奖者：**\\n1. **约翰·霍普菲尔德**（John J. Hopfield）— 美国普林斯顿大学教授\\n2.

杰弗里·辛顿（Geoffrey E. Hinton）— 加拿大多伦多大学教授\\n\\n**获奖理由：**\\n表彰他们

基于人工神经网络实现机器学习的基础性发现和发明。\\n\\n**主要贡献：**\\n-

约翰·霍普菲尔德：创造了一种联想记忆方法，能够存储和重构图像或其他类型的数据模式（即著名的**霍普菲尔德网络**）。\\n-

杰弗里·辛顿：发明了一种能够自动发现数据中属性的方法，可用于识别图片中的特定元素等任务（即**玻尔兹曼机**等相关技术）。\\n\\n**补充背景：**\\n-

两位获奖者利用**统计物理的基本概念**设计了人工神经网络，为当今强大的机器学习技术

奠定了基础。\\n-

他们的研究成果已广泛应用于粒子物理、材料科学、天体物理等研究领域，也渗透到日常生活中的**人脸识别**和**语言翻译**等

场景。\\n-

值得一提的是，杰弗里·辛顿还被称为"人工智能教父"，他此前也一直被认为是诺贝尔奖的热门候选人。\\n\\n有意思的是，这届物理

学奖颁发给了与计算机科学和人工智能相关的成果，这在诺贝尔物理学奖历史上较为特别。如果你对这两位科学家的研究细节或其

他年份的获奖者感兴趣，也可以继续问我哦！😊',

additional_kwargs={'refusal': None, 'reasoning_content':

''},

response_metadata={

'token_usage': {

'completion_tokens': 324,

'prompt_tokens': 3117,

'total_tokens': 3441,

'completion_tokens_details': {

'accepted_prediction_tokens': None,

'audio_tokens': None,

'reasoning_tokens': 0,

'rejected_prediction_tokens': None

},

'prompt_tokens_details': {'audio_tokens': None,

'cached_tokens': 1920},

'prompt_cache_hit_tokens': 1920,

'prompt_cache_miss_tokens': 1197

},

'model_provider': 'deepseek',

'model_name': 'deepseek-v4-flash',

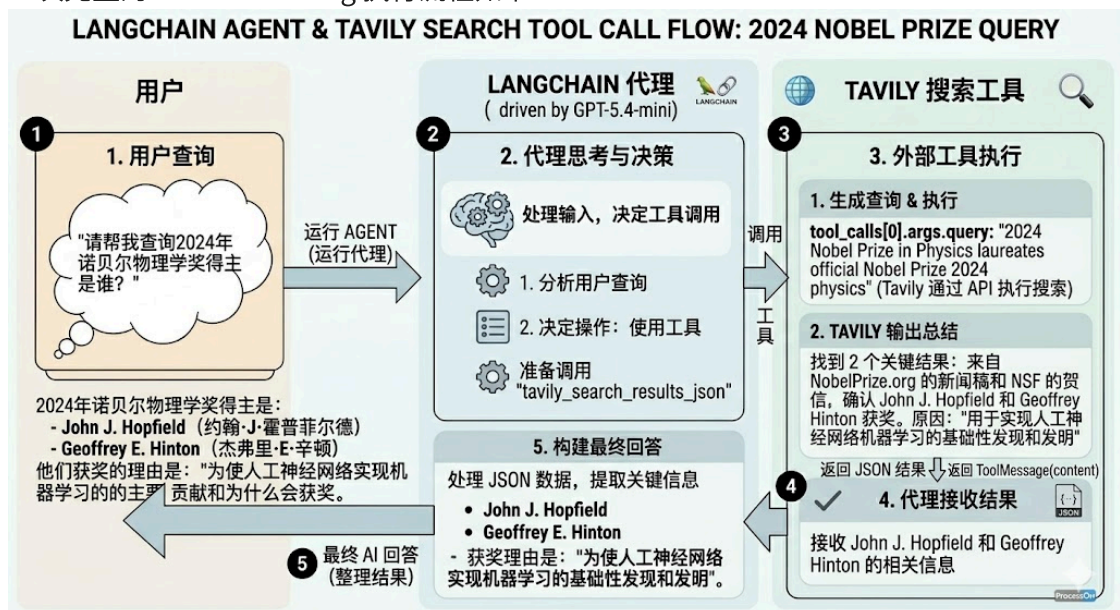

```

        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '73013dd3-07fd-405a-8f8a-86494cc9752d',
        'finish_reason': 'stop',
        'logprobs': None
    },
    id='lc_run--019ff3ed-eec1-7802-8302-8d8216e20eac-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 3117,
        'output_tokens': 324,
        'total_tokens': 3441,
        'input_token_details': {'cache_read': 1920},
        'output_token_details': {'reasoning': 0}
    }
}
    ]
}

```

如果仔细观察本次运行过程，本次工具调用仍然是一次典型的Function calling执行流程，包括用户首次发起消息在内，总共创建了4条消息，分别是human message、ai message（涉及function call message）、tool message（涉及function response message）以及ai message（涉及final responses）。

一次完整的Function calling 执行流程如下：



举例3：绑定多个工具

```

In [10]: from langchain.agents import create_agent
from langchain.tools import tool
from dotenv import load_dotenv
from rich import print as rprint

load_dotenv()

@tool(parse_docstring=True)
def get_weather(city: str):
    """
    天气查询工具
    """

```

```

Args:
    city: 城市名称
    """

    return f"{city}今天天气挺好"

@tool(parse_docstring=True)
def get_news():
    """
    新闻查询工具
    """

    return "近期，受全球储蓄芯片短缺等多重因素影响，多地回收商称废旧手机回收市场迎来“火热潮”，回收价格普遍上涨，旧手机成“香饽饽”。"

agent = create_agent(
    model=model,
    tools=[get_weather, get_news]
)

response = agent.invoke({
    "messages":["你好，杭州今天的天气如何？今天有哪些新闻？"]
})

rprint(response)

```

```

{
  'messages': [
    HumanMessage(
      content='你好，杭州今天的天气如何？今天有哪些新闻？',
      additional_kwargs={},
      response_metadata={},
      id='2df0024b-ebd1-4b1f-a00d-bc8bd0ed657f'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': "The user asks about weather in
Hangzhou and today's news. These are
independent calls, so I can make them in the same block."
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 88,
          'prompt_tokens': 397,
          'total_tokens': 485,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 28,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 397
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '2086d06b-ebbb-4d0b-871b-9bed62e417ad',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff3f8-18a4-7482-bd72-4ff09fd07c63-0',
      tool_calls=[
        {
          'name': 'get_weather',
          'args': {'city': '杭州'},
          'id': 'call_00_PoLHHMcSjdI0mAXxJGzc8694',
          'type': 'tool_call'
        },
        {'name': 'get_news', 'args': {}, 'id':
'call_01_DHZw2f1wdvM6Y7SM0gvo2469', 'type': 'tool_call'}
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 397,
        'output_tokens': 88,
        'total_tokens': 485,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 28}
      }
    ),
    ToolMessage(
      content='杭州今天天气挺好',
      name='get_weather',

```

```

        id='abc208b7-6d90-451a-b688-bf98f5718dc7',
        tool_call_id='call_00_PoLHMcSjdI0mAXxJGzc8694'
    ),
    ToolMessage(
        content='近期，受全球储蓄芯片短缺等多重因素影响，多地回收商称废旧手机回收市场迎来“火热潮”，回收价格普遍上涨，旧手机成“香饽饽”。',
        name='get_news',
        id='4034d583-c590-471a-9d31-9393c9ab489d',
        tool_call_id='call_01_DHZw2f1wdvM6Y7SM0gvo2469'
    ),
    AIMessage(
        content='为你查询到以下信息：\n\n**🌞 杭州天气**\n今天杭州天气不错，适合出行。\n\n**📰 今日新闻**\n近期受全球储蓄芯片短缺等多重因素影响，多地回收商表示废旧手机回收市场迎来“火热潮”，回收价格普遍上涨，旧手机成了“香饽饽”。\n\n如果还有其他需要查询的，随时告诉我~',
        additional_kwargs={'refusal': None, 'reasoning_content': ''},
        response_metadata={
            'token_usage': {
                'completion_tokens': 79,
                'prompt_tokens': 549,
                'total_tokens': 628,
                'completion_tokens_details': {
                    'accepted_prediction_tokens': None,
                    'audio_tokens': None,
                    'reasoning_tokens': 0,
                    'rejected_prediction_tokens': None
                },
                'prompt_tokens_details': {'audio_tokens': None,
            },
            'cached_tokens': 384},
            'prompt_cache_hit_tokens': 384,
            'prompt_cache_miss_tokens': 165
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'd93fd34a-7e1d-4783-87b0-465fc1a1cb5e',
        'finish_reason': 'stop',
        'logprobs': None
    ),
    id='lc_run--019ff3f8-1cb8-7c80-bce9-bedbdf5808f5-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 549,
        'output_tokens': 79,
        'total_tokens': 628,
        'input_token_details': {'cache_read': 384},
        'output_token_details': {'reasoning': 0}
    }
}
]
}

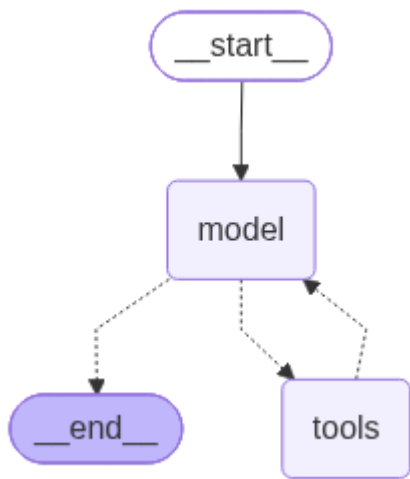
```

```

In [11]: from IPython.display import Image, display

display(Image(agent.get_graph().draw_mermaid_png()))

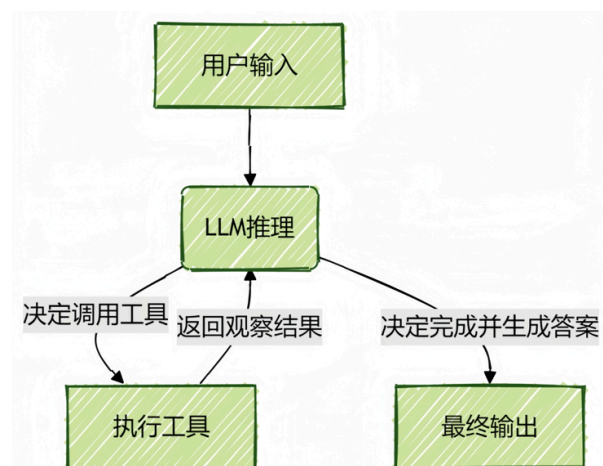
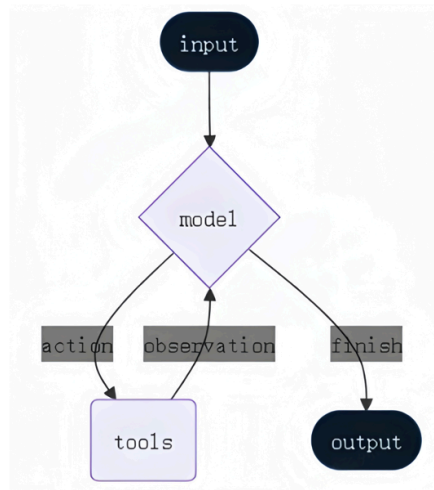
```



注意：只给 Agent 需要的工具，工具太多会混淆。一般2-5 个工具最佳

2) 工具调用流程分析

LangChain 的 Agent 会将模型与工具结合起来，在实现上由一个基于 LangGraph 的图结构来编排执行流程，如下所示。



这与前文得到的 Agent 图结构是一致的，本质上就是经典的 ReAct 结构：一个具备“思考-行动-观察”不断循环的自主工作者。

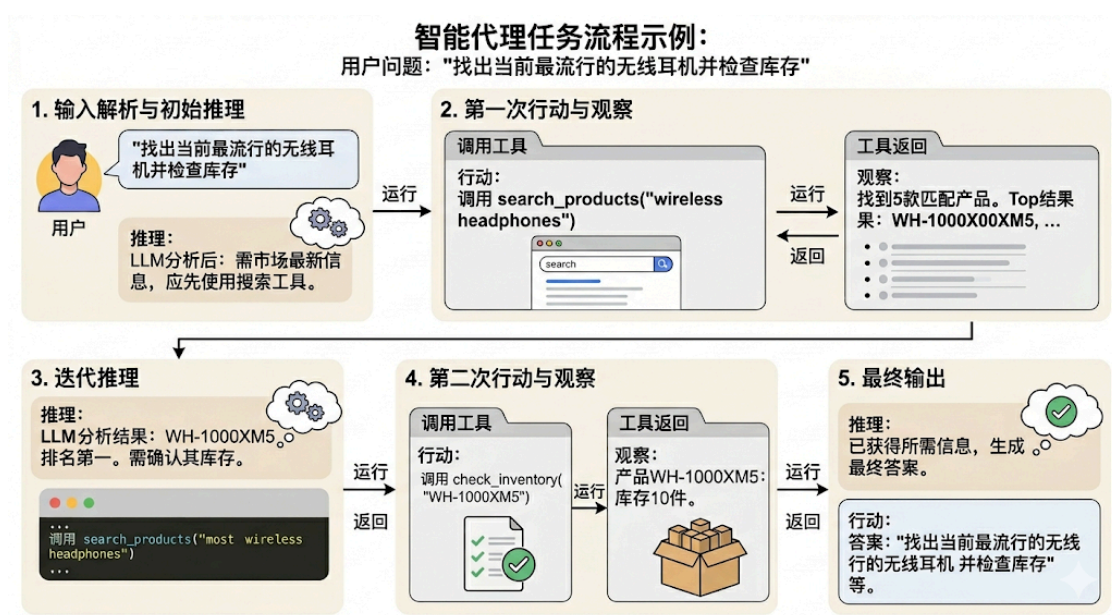
用户问题 → AI 思考 → 调用工具 → 观察结果 → 继续思考 → ... → 最终答案

当用户提出一个复杂需求时，Agent会像人类一样，先理解任务、规划步骤、使用合适的工具（如搜索网络、查询数据库、执行计算）获取信息，Agent 会在一个循环中反复调用模型和工具，直到某次模型输出中不再包含工具调用则结束，最后综合所有信息给出最终答案。

完整流程：



举例：用户问题：“找出当前最流行的无线耳机并检查库存”的任务。



1、输入解析与初始推理

- 输入：用户查询：“找出当前最流行的无线耳机并检查库存”。
- 推理：LLM分析任务后认为：“要找出‘最流行’的产品，需要最新的市场信息，我应该先使用搜索工具。”

2、第一次行动(action)与观察(observation)

- 行动：Agent调用search_products工具，参数为“wireless headphones”。
- 观察：工具返回结果：“找到5款匹配产品。Top结果：WH-1000XM5, ...”

3、迭代推理

- 推理：LLM根据搜索结果分析：“WH-1000XM5是排名第一的型号。现在需要确认其库存状态才能回答用户问题。”

4、第二次行动(action)与观察(observation)

- 行动：Agent调用check_inventory工具，参数为“WH-1000XM5”。
- 观察：工具返回：“产品WH-1000XM5：库存10件。”

5、最终输出

- 推理：LLM综合所有信息后认为：“已获得所需信息，可以生成最终答案。”
- 行动：模型生成最终答案，不再调用工具。

3) 重试机制

Agent可以在工具调用结果不满足要求时，自主重试。

```
In [12]: from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import SystemMessage, HumanMessage
from dotenv import load_dotenv
from rich import print as rprint

load_dotenv(override=True)
flag = 0
@tool
def get_weather(city: str):
    """
    天气查询工具

    Args:
        city: 城市名称
    """
    global flag
    flag += 1

    if flag < 3:
        # raise Exception("暂时无法访问")
        return "TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试"

    return f"{city}今天天气挺好"

messages = [
    SystemMessage("""
    你是一个天气助手。
    当工具返回以 'TEMP_UNAVAILABLE:' 开头的结果时，
    说明是临时故障，不要立即放弃；
    你应再次调用同一个工具，最多重试 3 次。
    如果 3 次后仍失败，再向用户说明服务暂时不可用。
    """),
    HumanMessage("你好，杭州今天的天气如何？")
]

agent = create_agent(model, tools=[get_weather])
response = agent.invoke({"messages": messages})
```

```
# rprint(response)

for msg in response["messages"]:
    msg.pretty_print()
```

===== System Message =====

你是一个天气助手。
当工具返回以 'TEMP_UNAVAILABLE:' 开头的结果时，
说明是临时故障，不要立即放弃；
你应再次调用同一个工具，最多重试 3 次。
如果 3 次后仍失败，再向用户说明服务暂时不可用。

===== Human Message =====

你好，杭州今天的天气如何？

===== Ai Message =====

Tool Calls:
get_weather (call_00_5iWwAsfNtVxEM0BWULD09364)
Call ID: call_00_5iWwAsfNtVxEM0BWULD09364
Args:
city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

天气服务返回了临时故障提示，我马上再试一次。
Tool Calls:
get_weather (call_00_iZr1S2FLesraBCGEMuua1819)
Call ID: call_00_iZr1S2FLesraBCGEMuua1819
Args:
city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

仍是临时故障，我继续重试。
Tool Calls:
get_weather (call_00_ET_QdNBPf5QHAD7NkjWFZ6p1107)
Call ID: call_00_ET_QdNBPf5QHAD7NkjWFZ6p1107
Args:
city: 杭州

===== Tool Message =====

Name: get_weather

杭州今天天气挺好

===== Ai Message =====

杭州今天天气挺好的！阳光和天气状况都不错，适合外出活动。祝你有愉快的一天！☀️

模型三次调用get_weather，最终获得了满意的结果。

4) 常见问题

问题1：Agent 如何选择工具？

依据：工具的 docstring

```
In [ ]: @tool
def get_weather(city: str) -> str:
    """获取指定城市的天气信息""" # ← AI Agent 读这个！
    ...

@tool
def calculator(operation: str, a: float, b: float) -> str:
    """执行基本的数学计算""" # ← AI Agent 也读这个！
    ...
```

AI 会根据：

1. 问题内容
2. 每个工具的描述
3. 自动选择最匹配的工具

问题2：Agent 为什么没有调用工具？

原因：

- 工具的 docstring 不清晰
- 问题表述不明确
- 模型认为不需要工具

解决：

```
In [ ]: # ❌ 不好
@tool
def tool1(x: str) -> str:
    """做一些事情""" # 太模糊

# ✅ 好
@tool
def get_weather(city: str) -> str:
    """
    获取指定城市的实时天气信息

    Args:
        city: 城市名称，如"北京"、"上海"
    """
```

问题3：Agent 选错工具？

原因：

- 多个工具的功能描述相似
- 工具太多导致混淆

解决：

- 只给必要的工具
- 工具描述要有明确区分
- 在 system_prompt 中说明工具使用场景

```
In [ ]: #  好：只给需要的工具
agent = create_agent(
    model=model,
    tools=[get_weather, calculator] # 2-5 个工具最佳
)

#  不好：工具太多
agent = create_agent(
    model=model,
    tools=[tool1, tool2, ..., tool20] # 会混淆
)
```

问题4：如何知道 Agent 何时完成？

当 AIMessage 不包含 tool_calls 时：

```
In [ ]: for msg in response['messages']:
    if isinstance(msg, AIMessage):
        if hasattr(msg, 'tool_calls') and msg.tool_calls:
            print("还在调用工具...")
        else:
            print("完成！最终答案：", msg.content)
```

问题5：Agent 可以调用多少次工具？

默认没有限制，直到得到最终答案。但可能会：

- 超时
- 达到 token 限制
- 模型决定停止

问题6：如何限制工具调用次数？

LangChain 1.0 的 create_agent 默认使用 LangGraph，可以通过配置限制：

```
In [ ]: # 注意：这是高级用法，后续会详细学习
config = {
    "recursion_limit": 5 # 最多 5 步
}

response = agent.invoke(input, config=config)
```

5.Agent的高级用法1：设置Agent名称

创建Agent时，LangChain允许用户指定其名称

5.1 用法

```
In [13]: from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os
from rich import print as rprint

# 从 .env 文件中加载环境变量
load_dotenv(override=True)

model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

agent = create_agent(
    model=model,
    name="chat_assistant"
)
response = agent.invoke({"messages": ["你好"]})

# rprint(response)
for msg in response["messages"]:
    msg.pretty_print()
```

===== Human Message =====

你好

===== Ai Message =====

Name: chat_assistant

你好！有什么可以帮你的吗？

输出的AI Message带有Name信息。

5.2 经典使用场景

name 在 Multi-Agent 场景中最常被提及，用于区分不同的 Agent。但它的作用并不局限于多 Agent 编排。在实际工程中，出现如下场景，通常都建议为 Agent 设置一个清晰且稳定的 name。

1. 流式输出归因

在启用流式输出时，name 可用于标识当前输出内容来自哪个 Agent。

这得多 Agent 协作、Agent 嵌套调用，或前端需要实时展示不同执行主体输出时尤其有用，便于准确区分 token 或事件的来源。

2. 消息身份标记

设置 name 后，Agent 产生的 AIMessage 会携带对应的name信息。这使得系统在保存会话记录、回放执行过程、构建审计日志或前端展示消息角色时，能够明确识别消息的生成者。

3. 调试与trace可读性

在调试、日志分析和链路追踪过程中，name 可以作为 Agent 的稳定标识，帮助开发者快速判断当前执行的是哪个 Agent。

当系统中存在多个能力相近的 Agent，或一个 Agent 被嵌套在更复杂的工作流中时，名称能够显著提升trace 的可读性和问题定位效率。

4. 组件化封装

在工程实践中，Agent 常被封装为可复用的能力模块，例如检索助手、SQL 助手、报告生成助手等。

为 Agent 设置 name，有助于在模块注册、运行监控、日志归档和能力复用时保持一致的身份标识。

如果后续需要将该 Agent 进一步作为子图节点、工具能力或子模块接入更复杂系统，也能降低维护和迁移成本。

5. 前端展示与运行态可观测性

在带有可视化界面的应用中，name 还可以直接作为运行时展示标识使用。

例如，在执行面板中显示“当前活跃 Agent”“本轮输出来源”或“调用链路中的执行节点”时，name 能帮助开发者和用户更直观地理解系统当前的执行状态。

6. 作为稳定的运行时身份标识

从更通用的角度看，name 可以理解为 Agent 在系统中的“运行时身份 ID”。

相比临时性的展示名称，一个稳定、规范的 name 更适合用于日志检索、监控统计、链路分析和跨模块协作，因此在生产环境中通常建议显式设置，而不是依赖默认行为。

6.Agent的高级用法2：系统提示词

使用 create_agent 创建 Agent 时，需传入模型和工具、可选地传入系统提示词。**提示词为 Agent提供了任务背景、行为准则和操作指南。**

系统指令，即SystemMessage，通过 system_prompt 设置，定义 Agent 行为。这个参数可以是str或者SystemMessage类型。

使用建议：

- 明确说明 Agent 的角色
- 定义输出格式
- 说明何时使用工具

比如：

```
In [ ]: agent = create_agent(  
        model=model,
```

```

tools=[get_weather],
system_prompt="""你是天气助手。
工作流程：
1. 理解用户的城市查询
2. 使用 get_weather 工具获取数据
3. 简洁清晰地回答

输出格式：
- 天气状况
- 温度
- 注意事项（如有）
"""
)

```

提示词设置有两种方式：静态设置和动态设置。动态设置需要借助中间件，后续讲解。

举例1：

```

In [14]: from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_tavily import TavilySearch
from dotenv import load_dotenv
import os
from rich import print as rprint

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

# 2. 工具实例化
web_search = TavilySearch(
    tavily_api_key=os.getenv("TAVILY_API_KEY"),
    max_results=2)

# 3. 创建Agent
agent = create_agent(
    model=model,
    tools=[web_search],
    system_prompt="你是一名多才多艺的智能助手，可以调用工具帮助用户解决问题。"
)

# 4. 运行Agent 获得结果
result = agent.invoke({
    "messages":[
        {"role":"user","content":"请帮我查询2024年诺贝尔物理学奖得主是谁？"}
    ]
})

# rprint(result)
print(result['messages'][-1].content)

```

根据查询结果，**2024年诺贝尔物理学奖**的获奖者是：

🏆 获奖者

1. **约翰·J·霍普菲尔德** (John J. Hopfield, 美国普林斯顿大学)
2. **杰弗里·E·辛顿** (Geoffrey E. Hinton, 加拿大多伦多大学)

🎯 获奖理由

他们因**"通过人工神经网络实现机器学习的基础性发现与发明"**而获奖。

📝 贡献简介

- **霍普菲尔德**：发明了以其名字命名的"霍普菲尔德网络" (Hopfield Network)，这是一种能够存储和重建信息的联想记忆模型，其物理思想深受自旋玻璃 (spin glass) 等统计物理模型的启发。
- **辛顿**：被誉为"人工智能教父"，他基于霍普菲尔德网络发展了**玻尔兹曼机** (Boltzmann machine)，为现代深度学习的发展奠定了理论基础。

值得一提的是，这两位科学家的获奖也引发了广泛讨论——因为这是物理学奖历史上首次授予与人工智能和机器学习直接相关的研究成果，标志着物理学方法（如统计力学）与计算机科学的深度融合。

需要我进一步介绍他们具体的科学贡献，或者查询其他年份的诺贝尔奖信息吗？

举例2：

```
In [15]: from langchain.agents import create_agent
from langchain_core.messages import SystemMessage
from langchain_core.tools import tool
from rich import print as rprint

# 工具：实现两数相加
@tool
def add_numbers(a: int, b: int) -> str:
    """计算并返回两个数的和。"""
    return f"和为：{a + b}"

# 创建客服助手Agent
agent = create_agent(
    model=model,
    tools=[add_numbers], # 工具列表
    # system_prompt="你是一个数学助手，解决日常的算术问题"
    system_prompt=SystemMessage(content="你是一个数学助手，解决日常的算术问题")
)
response = agent.invoke(
    {"messages": [
        {"role": "user", "content": "10加上20再加上30是多少？"}
    ]},
)

rprint(response)
# print(response["messages"][-1].content)
```

```

{
  'messages': [
    HumanMessage(
      content='10加上20再加上30是多少？',
      additional_kwargs={},
      response_metadata={},
      id='72a032de-87a8-47e2-a580-49506b7798bf'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'The user asks: 10 + 20 + 30 =
?\\n\\nI can use the add_numbers tool. But it
only takes two numbers at a time. I can add 10 and 20 first, then
add 30 to the result. These are dependent, so I
must call sequentially.\\n\\nLet me first add 10 + 20.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 130,
          'prompt_tokens': 382,
          'total_tokens': 512,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 70,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 382
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'd2584c67-db97-4b9e-9187-d308de0398b3',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff45b-649c-7920-8cab-c75b87050009-0',
      tool_calls=[
        {
          'name': 'add_numbers',
          'args': {'a': 10, 'b': 20},
          'id': 'call_00_mcTMZUcbqe7HsVnUnjem0663',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 382,
        'output_tokens': 130,
        'total_tokens': 512,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 70}
      }
    ),
    ToolMessage(
      content='和为：30',
      name='add_numbers',
      id='dd3c3fb3-951b-4216-8109-972aed3d995a',

```

```

        tool_call_id='call_00_mcTMZUcbqe7HsVnUnjem0663'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None, 'reasoning_content':
'Now add 30 to the result 30.'},
        response_metadata={
            'token_usage': {
                'completion_tokens': 70,
                'prompt_tokens': 527,
                'total_tokens': 597,
                'completion_tokens_details': {
                    'accepted_prediction_tokens': None,
                    'audio_tokens': None,
                    'reasoning_tokens': 10,
                    'rejected_prediction_tokens': None
                },
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512},
                'prompt_cache_hit_tokens': 512,
                'prompt_cache_miss_tokens': 15
            },
            'model_provider': 'deepseek',
            'model_name': 'deepseek-v4-flash',
            'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
            'id': '814ab25e-cee6-4086-88a7-c0fb87351ccf',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff45b-6abf-7791-9375-91c7f525f7c0-0',
        tool_calls=[
            {
                'name': 'add_numbers',
                'args': {'a': 30, 'b': 30},
                'id': 'call_00_TTSvv1kI3dICX1XLfc7k8310',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 527,
            'output_tokens': 70,
            'total_tokens': 597,
            'input_token_details': {'cache_read': 512},
            'output_token_details': {'reasoning': 10}
        }
    ),
    ToolMessage(
        content='和为：60',
        name='add_numbers',
        id='e8f908b9-887e-4f78-95f3-bffa62198f8e',
        tool_call_id='call_00_TTSvv1kI3dICX1XLfc7k8310'
    ),
    AIMessage(
        content='10 + 20 + 30 = **60**。 ',
        additional_kwargs={'refusal': None, 'reasoning_content':
''},
        response_metadata={
            'token_usage': {
                'completion_tokens': 13,
                'prompt_tokens': 612,
                'total_tokens': 625,
                'completion_tokens_details': {

```



```

        'accepted_prediction_tokens': None,
        'audio_tokens': None,
        'reasoning_tokens': 0,
        'rejected_prediction_tokens': None
    },
    'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512},
    'prompt_cache_hit_tokens': 512,
    'prompt_cache_miss_tokens': 100
},
    'model_provider': 'deepseek',
    'model_name': 'deepseek-v4-flash',
    'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
    'id': '4ff494b1-7859-4607-8fe7-44f06049674a',
    'finish_reason': 'stop',
    'logprobs': None
},
    id='lc_run--019ff45b-6ead-7ac2-9b49-b6b1ba24c7cf-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 612,
        'output_tokens': 13,
        'total_tokens': 625,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {'reasoning': 0}
    }
}
    )
}

```

举例3：

```

In [16]: from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import SystemMessage, HumanMessage

flag = 0

@tool
def get_weather(city: str):
    """
    天气查询工具

    Args:
        city: 城市名称
    """
    global flag
    flag += 1

    if flag < 3:
        # raise Exception("暂时无法访问")
        return "TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试"
    return f"{city}今天天气挺好"

messages = [
    HumanMessage("你好，杭州今天的天气如何？")
]
agent = create_agent(
    model=model,

```

```

tools=[get_weather],
system_prompt=SystemMessage(
    "你是一个天气助手。"
    "当工具返回以 'TEMP_UNAVAILABLE:' 开头的结果时，"
    "说明是临时故障，不要立即放弃；"
    "你应再次调用同一个工具，最多重试 3 次。"
    "如果 3 次后仍失败，再向用户说明服务暂时不可用。"
)
)

response = agent.invoke({"messages": messages})
# print(response)
for msg in response["messages"]:
    msg.pretty_print()

```

===== Human Message =====

你好，杭州今天的天气如何？

===== Ai Message =====

Tool Calls:

get_weather (call_00_zlrQR26x8jLm1CBxhXEL2194)

Call ID: call_00_zlrQR26x8jLm1CBxhXEL2194

Args:

city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

Tool Calls:

get_weather (call_00_MvUiGGZXOdHSUrZgpbRx3444)

Call ID: call_00_MvUiGGZXOdHSUrZgpbRx3444

Args:

city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

Tool Calls:

get_weather (call_00_QKD8ALxPTbyiRmGvRzFc0840)

Call ID: call_00_QKD8ALxPTbyiRmGvRzFc0840

Args:

city: 杭州

===== Tool Message =====

Name: get_weather

杭州今天天气挺好

===== Ai Message =====

杭州今天的天气挺好！☀️ 适合出门走走～

7.Agent的高级用法3：结构化输出

结构化输出是Agent的核心功能之一，它允许Agent以特定、可预测的格式返回数据，而不是传统的自然语言响应。通过结构化输出，开发者可以直接获得Pydantic模型、JSON对象或数据类等结构化数据，这些数据能够被应用程序直接使用，无需复杂的解析过程。